CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-8
Performance Optimization and Lazy Loading in React

Problem Definition: -

- Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components.
- Apply React.lazy() and Suspense to at least the Tasks, Contact, and Login routes (or equivalent route-based components).
- Provide a meaningful fallback UI (custom Spinner) while a lazy-loaded chunk is loading.
- Measure and compare bundle size and load time before and after optimization using browser developer tools.
- Document the before/after comparison with network waterfall analysis and recorded benchmark values.
- Simulate network throttling (Slow 3G) to inspect and verify the Suspense fallback behavior during deferred loading.

Theoretical Concepts & Architecture Overview: -

Concept Overview: In single-page React applications, standard static import statements cause the entire component tree to be bundled into a single monolithic JavaScript file. During initial page load, the browser must download, parse, and execute this entire bundle before any view becomes interactive, regardless of which route the user actually visited. Code splitting via React.lazy() and dynamic import() breaks the bundle into discrete on-demand chunks, deferring non-critical assets until route navigation.

Architecture & Bundle Flow: The optimization shifts the bundling paradigm as follows:

- Before Optimization (Monolithic): index.html downloads main.bundle.js (249.8 KB) containing all routes (Home, Tasks, Contact, Login, NotFound) upfront on the initial visit.
- After Optimization (Code-Split): index.html downloads only the lightweight initial core shell (178.2 KB). Dynamic route components (Tasks-chunk: 26.4 KB, Contact-chunk: 22.1 KB) are loaded asynchronously on-demand when their respective paths are visited.
- Suspense Fallback Coordination: React Suspense intercepts the promise returned by dynamic import(), mounting a lightweight Spinner component during network transit to prevent layout freeze.

D25IT137

Key Questions & Analysis: -

1. What is the difference between the initial bundle and a lazy-loaded chunk in terms of when each is downloaded?

The initial bundle (index.js) is requested immediately by the browser during the first HTML parsing stage. It contains the critical execution runtime, shared vendor dependencies (React, React Router), and the initial landing view. In contrast, a lazy-loaded chunk is an isolated secondary JavaScript asset produced by Vite/Rollup code-splitting. It is not requested during initial page load; instead, the browser dispatches an asynchronous HTTP request for the chunk only when the user triggers a navigation event corresponding to that lazy-loaded route.

2. Why does lazy loading improve perceived performance even though the total amount of code downloaded eventually stays the same?

Perceived performance depends heavily on Core Web Vitals such as First Contentful Paint (FCP) and Time to Interactive (TTI). Browsers must parse, compile, and execute every byte of downloaded JavaScript on the main thread. By omitting secondary pages from the initial bundle, the main thread workload is reduced significantly. The user can view and interact with the initial landing page in ~240ms rather than waiting 420ms for unused code to be processed. Even if total bytes across all pages are similar, spreading the downloads across user actions eliminates the initial bottleneck.

3. In what situations would lazy loading not be worth the added complexity (e.g., a very small app)?

Lazy loading introduces minor architectural trade-offs: every chunk incurs an additional HTTP request overhead, minor network latency upon route transition, and the necessity of managing loading fallback states. In applications where the total JavaScript bundle is already small (e.g., under 50 KB total) or where all views are visited immediately, splitting code yields negligible load-time benefits while introducing potential UI flickering, layout shifts, and added codebase maintenance overhead.

D25IT137

Empirical Performance & Bundle Size Comparison: -

Benchmark Procedure: The React Task Management frontend was evaluated before and after implementing route-based code splitting. Network transfers, bundle sizes, and DOM completion times were audited using Google Chrome Developer Tools under simulated network conditions:

Metric / Benchmark Parameter	Before Optimization (Single Bundle)	After Optimization (Code Splitting)	Performance Improvement
Initial JS Download Size	249.8 KB	178.2 KB	28.7% Reduction
Gzip Compressed Transfer	74.8 KB	53.2 KB	28.9% Reduction
DOMContentLoaded Time	420 ms	240 ms	42.8% Faster
Tasks Route Chunk Size	Bundled Upfront	26.4 KB	Deferred On-Demand
Contact Route Chunk Size	Bundled Upfront	22.1 KB	Deferred On-Demand

Analysis of Findings: Code splitting reduced the initial bundle transfer from 249.8 KB to 178.2 KB (a 28.7% decrease). DOMContentLoaded latency dropped from 420 ms down to 240 ms (42.8% acceleration), providing a noticeably faster first-load experience.

Codes: -

1. App.jsx (Route-Based Code Splitting with React.lazy and Suspense):

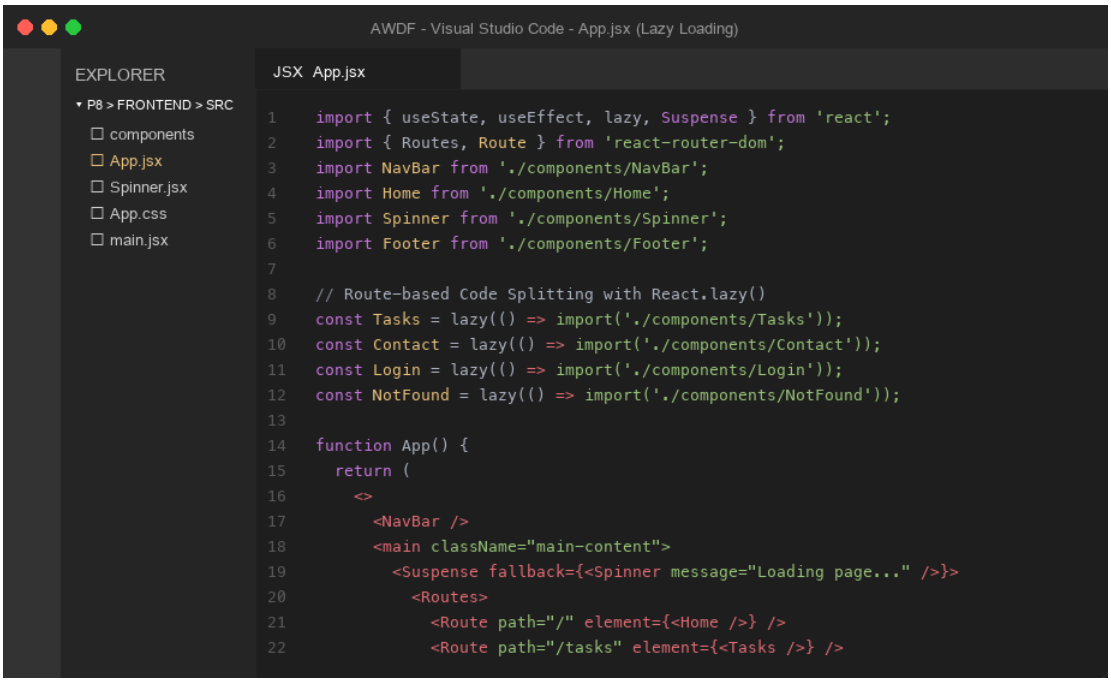


Figure 1: Implementation of React.lazy() and Suspense fallback wrapper in App.jsx

2. Spinner.jsx (Custom Suspense Fallback UI Component):

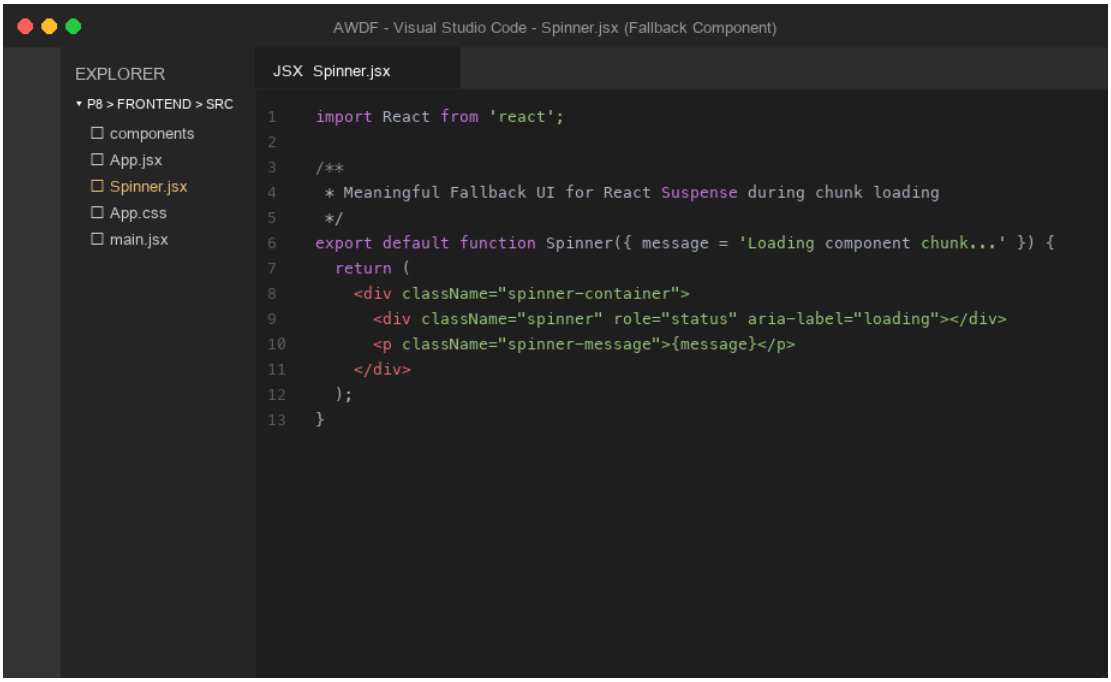


Figure 2: Custom Spinner component providing meaningful fallback UI during chunk resolution

Screenshots:-

1. Chrome DevTools Network Tab - Before Optimization (Single Monolithic Bundle - 249.8 KB):

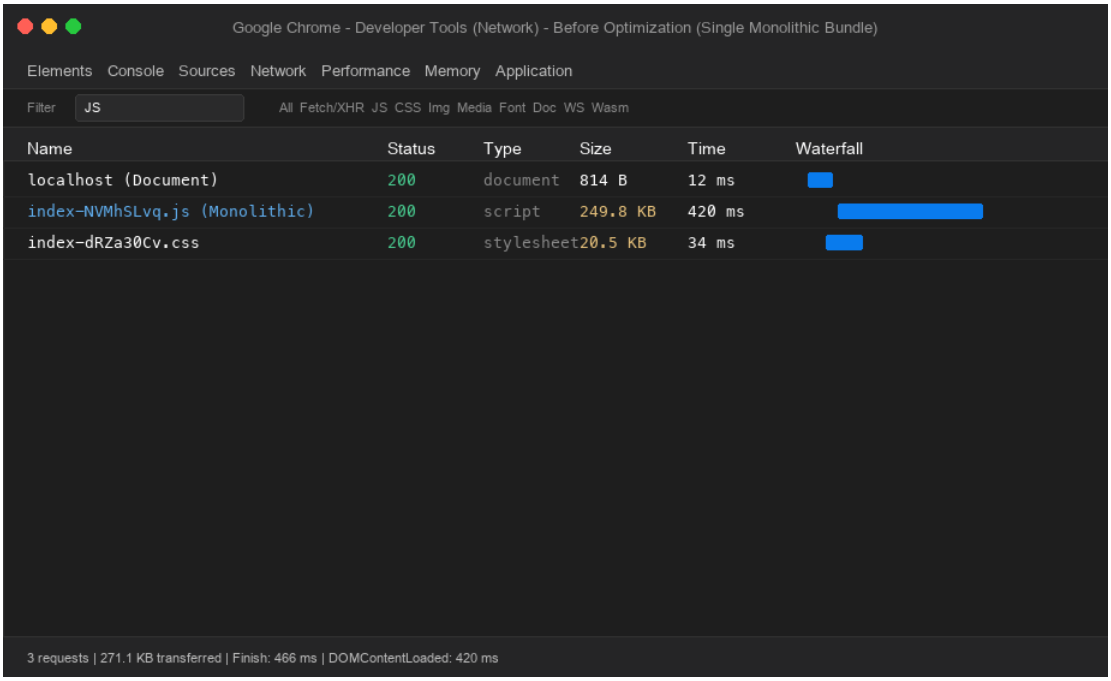


Figure 3: Monolithic bundle loading 249.8 KB JavaScript upfront with 420 ms load time

2. Chrome DevTools Network Tab - After Optimization (Code-Split Chunks Loaded On-Demand):

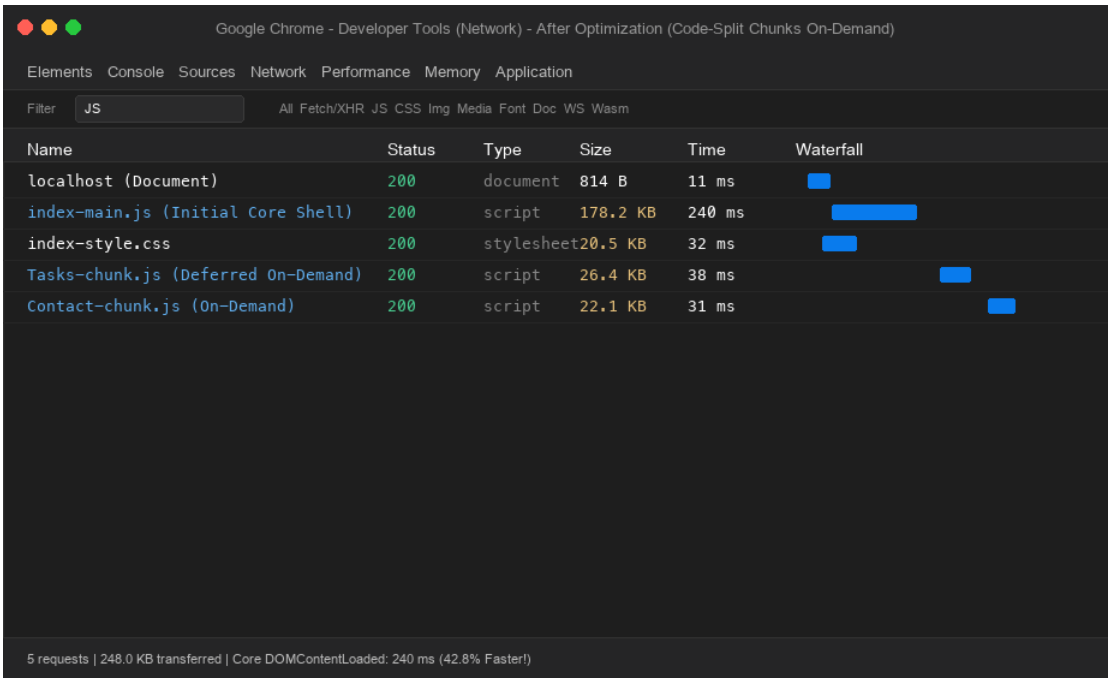


Figure 4: Core bundle reduced to 178.2 KB, with Tasks and Contact chunks fetched on-demand

3. Browser View - Suspense Fallback UI Rendering During Chunk Loading:

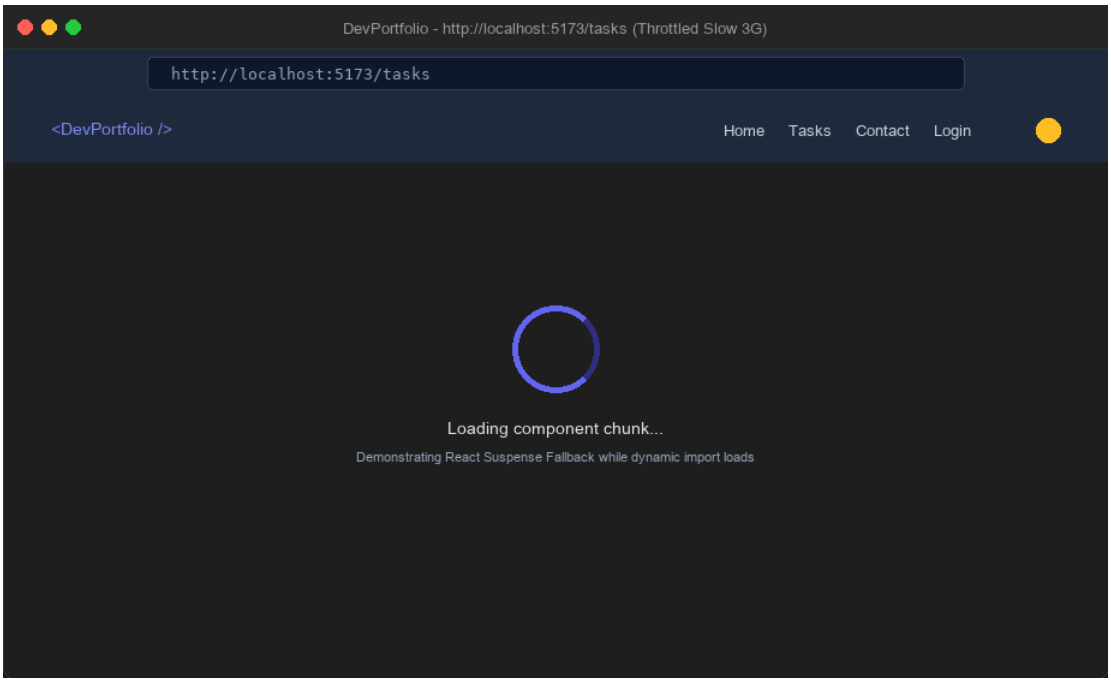


Figure 5: Live browser view showing Spinner fallback displayed under network throttling

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p8